

Navigators Technical Documentation

Navigators — Reproducibility & Replication Guide

Document Identifier: REP-NAV-01
Status: Production Reference
Version: 1.0.0
Last Updated: 2026-09-23
Attribution: Developed by Navigators

1. System Replication Standard

This guide provides exact, step-by-step instructions to reproduce the entire Navigators pipeline: from downloading the benchmark dataset, running data preparation, training the TCN neural model, exporting to ONNX, verifying PyTorch-to-ONNX parity, to executing the full seven-stage ablation benchmark suite.

2. Step 1: Environment Installation & Verification

```
# 1. Clone repository
git clone https://github.com/Navigators/Navigators.git
cd Navigators

# 2. Setup virtual environment
python3 -m venv venv
source venv/bin/activate

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Verify baseline automated test suite (288 tests)
./venv/bin/pytest tests
node --test tests/vehicle_motion_detection.test.cjs
node --test tests/offline_navigation.test.cjs
```

Expected Outcome: All 288 automated tests pass with zero failures.

3. Step 2: Download the Official IO-VNBD Benchmark

```
# 1. Ensure Git LFS is installed
brew install git-lfs # macOS (or sudo apt install git-lfs on Linux)
git lfs install

# 2. Clone IO-VNBD directly into data/IO-VNBD
cd data
git clone https://github.com/onyekpeu/IO-VNBD.git
cd IO-VNBD
git lfs pull
cd ../../
```

4. Step 3: Data Preparation & Session Splitting

Execute the data pipeline to parse synchronized sensor and vehicle CSV pairs into session-level train, validation, and test splits:

```
python -c "
from src.models.iovnbd_dataset import create_iovnbd_data loaders
train_l, val_l, test_l = create_iovnbd_data loaders('data/IO-VNBD', window_size=200,
    batch_size=64)
print(f'Train batches: {len(train_l)}, Val batches: {len(val_l)}, Test batches: {len(test_l)}')
"
```

Expected Outcome: Train sessions=100 (63,423 windows), Val sessions=22 (30,349 windows), Test sessions=22 (11,934 windows).

5. Step 4: Model Training

Train the production TCN model:

```
python scripts/train.py --config configs/default.yaml
```

- Parameters: 5,400,322 weights.
 - Optimizer: AdamW, Initial LR: 0.001, Weight Decay: 0.0001.
 - Output checkpoint saved to: `checkpoints/best_model.pt`.
-

6. Step 5: Held-Out Test Evaluation

Evaluate the trained checkpoint against the held-out 22 test sessions:

```
python evaluate_model.py
```

Outputs mean absolute error (MAE in m/s and km/h), root-mean-squared error (RMSE), and baselines (Zero Velocity MAE, Mean Velocity MAE).

7. Step 6: ONNX Export & Parity Audit

Export the PyTorch model to ONNX format and audit mathematical parity:

```
# Export and verify parity on real input window tensors:
python scripts/verify_onnx.py --dataset data/raw_trips/trip_4.json --export --stats
                                checkpoints/norm_stats.json
```

Expected Outcome: Parity test passes with absolute tolerance $\text{atol} \leq 10^{-5}$ and relative tolerance $\text{rtol} \leq 10^{-4}$.

8. Step 7: Seven-Stage Ablation Replay

Run the complete A through G ablation benchmark suite across an identical simulated GNSS outage:

```
python replay.py --dataset data/raw_trips/trip_4.json --gnss-outage 60 --outage-start 10 --
                ablations
```

- Generates `results/replay/trip_4_results.json` and `results/replay/trip_4_ablation.csv`.
 - Generates individual trajectory CSVs for all modes: `trip_4_A_trajectory.csv` through `trip_4_G_trajectory.csv`.
-

Developed by Navigators